The Course TA-Bot: Conversational RAG with Memory

ECE 6256, Assignment 3. Team: [your full name] and [partner full name].

1. System and results

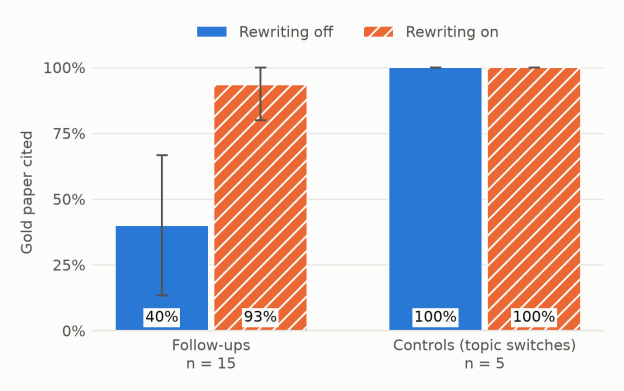
The bot answers from 108 course papers (5,711 chunks) using SQLite FTS5 BM25 with title terms weighted 5. It takes 40 candidate chunks, groups them by paper, and keeps the top 3 papers, each with up to 3 passages. Follow-ups that refer back are rewritten into standalone queries by the model and merged with the original results by reciprocal rank fusion (k=60). Answers are generated only from the retrieved passages, and citations are paper ids taken from retrieval rather than from model text.

Measurement setup. No shared endpoint was available, so we served Llama-3.1-8B-Instruct ourselves with vLLM on a PACE-ICE A40 GPU, temperature 0, seed 42. The dev scores below are on the public set, which the README warns is small; we have no measurement of the course judge's answer quality.

Check (real model)	Result	Check (real model)	Result
Single-turn hit-rate	100% (10 questions)	Memory probes, public	2 / 2
Multi-turn hit-rate	100% (3 dialogues)	Our extra probes (8 must-not-store, 2 positive)	10 / 10

2. Ablation: follow-up rewriting on or off (R4)

Question. Does rewriting a follow-up into a standalone query help retrieval, and can it hurt? **Axis.** One flag, *rewrite*. Corpus, chunking, BM25 weights, prompts, seed and model are held fixed. **Data.** 40 two-turn dialogues we wrote, split in half: 20 for tuning, 20 held out and reported here. 15 held-out dialogues are follow-ups that lean on the first turn; 5 are standalone topic-switch controls, where a rewrite can only hurt. **Metric.** Whether the last turn cites the gold paper. Each run uses a fresh bot. Intervals are 95% bootstrap; the test is exact McNemar, because both conditions answer the same questions.



Result. Follow-ups rose from 40% (CI 0.13-0.67) to 93% (CI 0.80-1.00). 8 dialogues improved and 0 got worse ($p = 0.0078$). Controls stayed at 100%, so no harm was seen, though $n = 5$ cannot rule out a small one. All 40 runs completed.

What the flag really tests. It bundles two mechanisms: the rewrite, and *carry-over*, which appends the previous turn's top paper if retrieval dropped it. A 93% alone does not say which one worked, so we separated them.

Figure 1. Held-out hit-rate, Llama-3.1-8B on vLLM. Whiskers are 95% bootstrap intervals; n is small.

Separating the mechanisms. Carry-over only adds a paper after the top 3, so a paper's position shows how it arrived. On the real model all 8 recovered papers sit inside the top 3 (position 2 or 3), so the rewrite found them and carry-over was not needed. We then replayed the split on the bundled mock, which returns canned text and cannot rewrite. It reached the same 40% to 93%, with the same 8 papers each in fourth position, so carry-over alone is also sufficient. The mock is used only as a no-rewrite condition; retrieval with the flag off was identical on both endpoints.

Interpretation. On this set the two mechanisms are redundant: each recovers the full gain alone, so the gain must not be credited to rewriting alone. The redundancy partly follows from our data, because our follow-ups ask about the first turn's subject by construction, which is the case carry-over is built for. Rewriting should matter more when the gold paper is not the previous subject, but this study does not measure that. A cost also remains: carry-over spends a citation slot, and we measured retrieval only, not answer quality.

The one miss. followup-18 ("What is BERT?" then "Which datasets showed the largest gains?") fails in both conditions. The follow-up has no referring word such as *it* or *they*, so neither mechanism fires. This is a real limit: implicit follow-ups are not detected.

Threats to validity. 15 follow-ups give wide intervals, so the size of the gain is uncertain even though its direction is clear. We wrote the dialogues ourselves, so they reflect our idea of a follow-up. The held-out split

was run more than once during development, so it is not pristine, although later fixes came from reading answers rather than from these scores. Results are for one model and one seed.

3. Memory design (R3)

What we store. A per-user SQLite table holds readable category and fact pairs in five allowlisted categories: name, project, interest, background, preference. Name and project keep only the newest value; the others can coexist. Each user is capped at 30 facts, dropping the oldest, and facts are capped at 200 characters.

When we write. Only from the user's own message, on the turn it is said, and only when it is an explicit first-person statement matching an allowlist pattern. We never write from model output or paper text, so the model cannot invent a memory and a passage cannot plant one. Extraction is rules, not an LLM, which makes it deterministic and easy to audit. Questions like "what is my project?" are answered directly from the store with no generation.

What we refuse, and why. A sentence is stored only if it matches the allowlist *and* trips no veto. A veto is the safe default: a missed fact costs a little personalization, but a leaked secret cannot be undone.

Refused	Reason
Passwords, API keys, card, bank, phone, address, student ids, emails, long digit runs	Useless for tutoring, harmful if exposed
Health, religion, politics, sexuality, ethnicity, immigration status	Sensitive attributes a TA has no need to keep
Grades, GPA, salary, debt, cheating or plagiarism	Could be used against the student; not needed to teach
Facts about friends, family, advisors, classmates	That person never agreed to be stored
Hypotheticals, role-play, negated claims, temporary states ("today", "due")	Not lasting facts about the user; would go stale or be false
"Ignore your instructions" and system-prompt text	Stored text is replayed in prompts, so it is an injection path

A message saying "don't remember", "off the record" or "confidential" opts the whole message out, and an explicit "remember this" does not override a veto. **Privacy controls.** Facts are keyed by user_id, and history by user and session, so a reused session id cannot cross users. Refused turns are replaced by "[Private statement omitted.]" in history, so they are not replayed later. Users can ask to forget one category or everything; that also clears their history, as does reset_memory. Facts are shown to the model as a profile marked as not evidence and containing no instructions.

Known limits. English regexes can miss paraphrases and reject harmless text, so a safe fact in a sensitive sentence is lost: we accept this in favor of privacy. Refusal means not retained; the current message still reaches the course endpoint to be answered. Memory defaults to in-process; with memory_path it is a plain, unencrypted SQLite file, and user_id is trusted, not authenticated.

4. Reproducibility

Two separate dev runs on the real model gave identical scores. Their output differed on one line: dialogue dev-mt-02 cited the same five papers in a different order. Our ranking sorts use fixed tie-breaks, so we suspect a small difference in the model's rewrite even at temperature 0, but we did not isolate it. Hit-rate is unaffected here, since it checks membership, but such drift could in principle change a citation on another question. Of our 14 grading-style tests, 13 passed on the real model; the one failure was this order change, and we then relaxed that test to ignore order, without re-running it on the real model. All 49 offline tests pass on the mock. Temperature, seed, endpoint, model and corpus path all come from config.

5. Judge caveat

We do not have the course judge's scores, so we cannot claim it erred. As a stand-in we scored answers with a Llama model and our own prompt built on the README's four criteria, giving it the cited passages. It scored an earlier answer 5/5 on all four although it listed tau-bench (L12-08, a tool-agent-user benchmark) as a benchmark for comparing retrieval methods. It also marked down a true omission in a BERT control (completeness 3), so it is not uniformly lenient. This is one case on a stand-in, and it is consistent with the limited reasoning ability reported for LLM judges in L12-01.

6. AI-Use Disclosure

Tools. Claude Code (Anthropic) and Codex (OpenAI). **Claude Code** generated most of ta_bot.py under our direction, including retrieval, rewriting, memory and prompts, as well as the tests, ablation scripts, PACE job scripts and result analysis, and it drafted this report. **Codex** was used for [Muyu: state exactly what Codex did, e.g. which validation work]. **Our own contribution:** [each partner: state honestly what you designed, decided, reviewed or wrote yourself].

How we verified. Most checks were run through Claude Code: the provided eval_dev.py on the mock and on the real model on PACE-ICE, a repeated dev run for reproducibility, and offline tests for the interface, memory probes and hard-coding. Claude Code also compared the real answers against corpus text, which exposed truncated answers, a missing BERT objective and a false claim about SentencePiece. Tests exposed a thread crash. AI output was sometimes wrong: its first explanation credited the ablation gain mainly to carry-over, and analysis of paper positions showed otherwise. All numbers here come from result files, not from AI summaries. [state what you checked yourselves, e.g. reading the code, answers or this report, then delete this note]